

Software Documentation for SnapTray

Table of Contents

- [Software Documentation for SnapTray](#)
 - [Table of Contents](#)
 - [1. Introduction](#)
 - [2. System Overview](#)
 - [3. Requirements Specification](#)
 - [4. System Architecture](#)
 - [4.1 Components/Modules](#)
 - [4.2 Data Flow](#)
 - [4.3 Technologies](#)
 - [5. Design](#)
 - [5.1 Design Principles](#)
 - [5.1.1 Modularity and Separation of Concerns](#)
 - [5.1.2 Security-First Approach](#)
 - [5.1.3 Scalability and Performance](#)
 - [5.1.4 User-Centric Design](#)

- 5.1.5 Reliability and Fault Tolerance
- 5.1.6 Maintainability and Extensibility
- 5.1.7 Data Integrity and Consistency
- 5.1.8 Cross-Platform Compatibility
- 5.2 Database Design
 - 5.2.1 Az adatbázis célja, funkciója és a benne tárolt információk összefoglalása
 - 5.2.2 Adatbázis-terv és séma
 - 5.2.2.1 Entitások és kapcsolatok (ER modell összefoglaló)
 - 5.2.2.2 Relációs séma (táblázatok részletei)
 - 5.2.2.2.1 User (Felhasználók)
 - 5.2.2.2.2 Payment (Kifizetések)
 - 5.2.2.2.3 MenuItems (Menüelemek)
 - 5.2.2.2.4 Order (Rendelések)
 - 5.2.2.2.5 OrderItems (Rendelés tételek)
 - 5.2.2.2.6 Review (Értékelések)
 - 5.2.2.2.7 DailyMenu (Napi menü)
 - 5.2.2.2.8 ParentStudent (Szülő-Diák kapcsolat)
 - 5.2.2.2.9 SecurityLogs (Biztonsági naplók)
 - 5.2.2.2.10 UserLoyalty (Hűségprogram)
 - 5.2.2.3 Fizikai és logikai szerkezet

- 5.2.2.4 Használati esetek (Use Cases) és forgatókönyvek
- 5.2.2.5 Biztonság és hozzáférés
- 5.2.2.6 Karbantartás és üzemeltetés
- 5.3 Algorithms and Data Structures
 - 5.3.1 Data Structures
 - 5.3.1.1 MongoDB Collections (NoSQL Documents)
 - 5.3.1.2 Redis Data Structures
 - 5.3.1.3 Redis Lua Scripting
 - 5.3.1.4 JavaScript Objects and Arrays
 - 5.3.2 Core Algorithms
 - 5.3.2.1 Authentication and Security Algorithms
 - 5.3.2.2 Payment Processing Algorithms
 - 5.3.2.3 Loyalty Point Calculation Algorithm
 - 5.3.2.4 Search and Filtering Algorithms
 - 5.3.2.5 Caching Algorithms
 - 5.3.2.6 Rate Limiting Algorithm
 - 5.3.2.7 Data Validation Algorithms
 - 5.3.3 Performance Optimization Algorithms
 - 5.3.3.1 Database Query Optimization
 - 5.3.3.2 Pagination Algorithm

- 5.3.3.3 Batch Processing Algorithm
- 5.3.4 Security Algorithms
 - 5.3.4.1 reCAPTCHA Verification
- 5.3.5 Algorithm Selection Rationale
- 5.4 Security Design
- 6. Implementation
- 7. Testing and Validation
- 8. User Manual
- 9. Deployment and Maintenance
- 10. Conclusion and Future Work
- 11. References
- 12. Appendices

1. Introduction

2. System Overview

3. Requirements Specification

4. System Architecture

MERN Stack with Redis + Lua Architecture Repository: bivorkz/iskolaprojekt

Client Layer

- React Frontend

Application Layer (Node.js)

- Express Server (main.js)
- Dashboard Routes
- Payment Gateway
- Authentication
- API Endpoints
- Order Processing

Service Layer

- Loyalty Service
- Redis Lua Service (redis-lua-service.js)
- Cache Service (cache-service.js)
- Script Loader (script-loader.js)

Database Layer

- MongoDB (NoSQL)
 - Collections: Users, Orders, Payments, MenuItems, Loyalty Programs
- Redis (In-Memory DB)
 - Rate Limiting
 - Session Store
 - Cache Storage
 - Wallet Balances

Data Flow and Interactions:

- React Frontend sends HTTP Requests (REST API) to Express Server (main.js).
- Express Server (main.js) interacts with:
 - Dashboard Routes (Fetch User Data, Store Payments, User Authentication)
 - Payment Gateway (Atomic Operations, Cache Queries)
 - Authentication (Transaction Locks)
 - API Endpoints (Rate Limiting, Session Validation)
 - Order Processing (Process Orders, Inventory Check)
- Express Server (main.js) interacts with Redis (In-Memory DB) for:
 - Rate Limiting
 - Session Validation
 - Process Orders
 - Inventory Check
- Express Server (main.js) interacts with MongoDB (NoSQL) for:
 - Fetch User Data
 - Store Payments
 - User Authentication
 - Save Order Data
 - Loyalty Points
- Express Server (main.js) interacts with Service Layer for:
 - Loyalty Service
 - Redis Lua Service (redis-lua-service.js)
 - Cache Service (cache-service.js)
- Redis Lua Service (redis-lua-service.js) interacts with Redis (In-Memory DB) for:
 - Execute Lua Scripts (EVALSHA)
 - Store/Retrieve Cached Data
 - Load Lua Scripts at Initialization
- Cache Service (cache-service.js) interacts with Redis (In-Memory DB) for:
 - Cache Storage
 - Wallet Balances
- Script Loader (script-loader.js) interacts with Redis (In-Memory DB) for:
 - Load
 - Load
 - Load
- Redis (In-Memory DB) interacts with Lua Scripts for:
 - process_order.lua (Execute Order Processing Atomically)
 - rate_limit.lua (Execute Rate Limit Logic Atomically)
 - wallet_update.lua (Execute Wallet Updates Atomically)

Redis Purposes:

- Rate limiting (sliding window)
- Session management
- Cache layer (300s TTL)
- Atomic operations via Lua
- Real-time wallet balances

MongoDB Collections:

- Users (authentication)
- Orders (history)
- Payments (transactions)
- MenuItems (catalog)
- Loyalty programs

process_order.lua
Inventory + wallet check atomically in Redis

rate_limit.lua
Sliding window algorithm using ZADD/ZREMRANGEBYSCORE

wallet_update.lua
Atomic balance updates with validation

Data Flow:

- Client -> Express Server
- Redis cache check first
- Rate limiting via Lua
- Atomic operations in Redis
- Persistent data in MongoDB

Stack:

MongoDB + Express + React + Node.js + Redis + Lua

Data Flow:

1. Client → Express Server
2. Redis cache check first
3. Rate limiting via Lua
4. Atomic operations in Redis
5. Persistent data in MongoDB

Stack:
MongoDB + Express + React
+ Node.js + Redis + Lua

4.1 Components/Modules

The SnapTray system is organized into several key subsystems that work together to provide a cohesive cafeteria management solution:

- **Frontend/UI Layer:** Built with React.js, this layer handles user interactions, displays the interface, and manages client-side state. It includes components for authentication, menu browsing, ordering, and dashboard management.
- **API Layer:** Implemented using Node.js and Express.js, this layer provides RESTful endpoints for data access and business logic execution. It handles requests from the frontend, processes them, and returns appropriate responses.
- **Data Layer:** Comprises MongoDB for persistent data storage and Redis for caching and session management. This layer ensures data integrity, efficient retrieval, and temporary storage for performance optimization.
- **Authentication and Security Module:** Integrated across all layers, this module manages user authentication, authorization, rate limiting, and security logging.
- **Payment Processing Module:** Handles integration with external payment providers like PayPal and Google Pay, ensuring secure and reliable transaction processing.
- **Loyalty System Module:** Manages user points, tiers, and discounts, calculating rewards based on purchase history and applying appropriate benefits.

4.2 Data Flow

Data flows through the SnapTray system in a structured manner to ensure security, performance, and consistency:

1. **User Interaction:** Users interact with the React frontend, triggering API calls via HTTP requests.
2. **Request Processing:** The Express server receives requests, validates them through middleware (authentication, rate limiting, input sanitization), and routes them to appropriate handlers.

3. **Business Logic Execution:** Handlers execute business logic, often involving database queries to MongoDB or cache checks in Redis.
4. **Data Retrieval/Storage:** For read operations, data is fetched from Redis cache if available, otherwise from MongoDB. Write operations update both database and invalidate relevant cache entries.
5. **Response Generation:** Processed data is formatted and sent back to the frontend as JSON responses.
6. **Real-time Updates:** For certain operations (like wallet balance changes), Redis pub/sub is used to notify connected clients of updates.
7. **Logging and Monitoring:** All significant actions are logged to SecurityLogs collection for audit and monitoring purposes.

This flow ensures that sensitive operations are atomic, cached data remains consistent, and the system can handle concurrent users efficiently.

4.3 Technologies

The technology stack for SnapTray was chosen to balance development speed, performance, scalability, and maintainability:

- **MERN Stack (MongoDB, Express.js, React.js, Node.js):** Selected for its JavaScript full-stack consistency, reducing context switching and enabling shared code between frontend and backend. MongoDB's document-based structure aligns well with the flexible data requirements of a cafeteria system.
- **Redis:** Chosen for its high-performance in-memory data structure store, ideal for caching frequently accessed data (user sessions, menu items, statistics) and implementing features like rate limiting and real-time notifications. Its atomic operations ensure data consistency in high-concurrency scenarios.
- **Lua Scripting in Redis:** Utilized for executing complex, atomic operations on Redis data structures. Chosen because it allows multiple Redis commands to run as a single, uninterruptible transaction, ensuring data consistency, preventing race conditions, and improving performance by reducing network round trips between the application and Redis server.
- **Tailwind CSS:** Selected for rapid UI development with utility-first approach, ensuring responsive design and consistent styling across devices.

- **JWT for Authentication:** Provides stateless authentication, reducing server-side session storage needs and improving scalability.
 - **bcrypt for Password Hashing:** Industry-standard for secure password storage, protecting against rainbow table and brute-force attacks.
 - **reCAPTCHA:** Integrated to prevent automated abuse while maintaining user experience.
 - **PayPal/Google Pay APIs:** Chosen for their robust security, global acceptance, and ease of integration for payment processing.
 - **Render:** Used for deployment due to its simplicity, scalability, and support for modern web applications.
 - **Git/GitHub:** For version control and collaborative development.
 - **IPlocate.io:** Integrated for IP geolocation services to enhance security logging and fraud detection.
- The architecture follows a monolithic approach rather than microservices due to the project's scope and team size, allowing for simpler deployment, debugging, and data consistency. Horizontal scaling is achieved through MongoDB sharding and Redis clustering when needed.

5. Design

5.1 Design Principles

The SnapTray system follows several key design principles to ensure a robust, scalable, and user-friendly school cafeteria management solution. These principles guide the architecture, implementation, and maintenance of the system.

5.1.1 Modularity and Separation of Concerns

- **Component-based Architecture:** The system is built using React components for the frontend, ensuring reusable and maintainable UI elements.

- **Service Layer Abstraction:** Business logic is separated into dedicated service modules (order-service.js, paypal-service.js, etc.) to maintain clean separation between data access, business rules, and presentation layers.
- **Middleware Organization:** Security, authentication, and validation logic are implemented as Express middleware, allowing for clean request processing pipelines.

5.1.2 Security-First Approach

- **Defense in Depth:** Multiple layers of security including input validation, authentication, authorization, rate limiting, and data sanitization.
- **Principle of Least Privilege:** Users have access only to the minimum resources necessary for their role (student, parent, teacher, admin).
- **Secure by Default:** All endpoints require authentication by default, with explicit permissions granted for specific operations.
- **Data Protection:** Sensitive data (passwords, payment information) is encrypted and hashed appropriately.

5.1.3 Scalability and Performance

- **Horizontal Scaling:** The system uses MongoDB and Redis, which support horizontal scaling through sharding and clustering.
- **Caching Strategy:** Redis is employed for session management and data caching to reduce database load and improve response times.
- **Asynchronous Processing:** Non-blocking I/O operations and asynchronous programming patterns ensure the system can handle concurrent users efficiently.
- **Resource Optimization:** Database queries are optimized with proper indexing, and large datasets are paginated.

5.1.4 User-Centric Design

- **Intuitive User Interface:** Clean, responsive design using Tailwind CSS that works across devices (desktop, tablet, mobile).
- **Progressive Enhancement:** Core functionality works without JavaScript, with enhanced features for modern browsers.

- **Accessibility:** Following WCAG guidelines to ensure the system is usable by students with disabilities.
- **Feedback Systems:** Clear error messages, loading states, and success confirmations to guide users through processes.

5.1.5 Reliability and Fault Tolerance

- **Error Handling:** Comprehensive error handling with graceful degradation - the system continues to function even when individual components fail.
- **Transaction Management:** Database operations use transactions where appropriate to maintain data consistency.
- **Logging and Monitoring:** Extensive logging for debugging and monitoring system health, with different log levels for different environments.
- **Backup and Recovery:** Regular database backups and recovery procedures to prevent data loss.

5.1.6 Maintainability and Extensibility

- **Clean Code Principles:** Following established coding standards, meaningful variable names, and comprehensive documentation.
- **Version Control:** Git-based development with feature branches and proper commit messages.
- **API Design:** RESTful API design with consistent naming conventions and response formats.
- **Configuration Management:** Environment-based configuration to support different deployment environments (development, staging, production).

5.1.7 Data Integrity and Consistency

- **Validation Layers:** Input validation at multiple levels (client-side, server-side, database-level).
- **Referential Integrity:** Proper use of MongoDB references and population to maintain relationships between entities.
- **Atomic Operations:** Critical operations (like payment processing and loyalty point updates) use atomic database operations.

- **Audit Trail:** Security logs track all important actions for compliance and debugging.

5.1.8 Cross-Platform Compatibility

- **Browser Support:** Compatible with modern browsers (Chrome, Firefox, Safari, Edge) with graceful degradation for older browsers.
- **Mobile Responsiveness:** Responsive design ensures usability on various screen sizes.
- **API Flexibility:** RESTful APIs that can be consumed by different clients (web, mobile apps, third-party integrations).

5.2 Database Design

5.2.1 Az adatbázis célja, funkciója és a benne tárolt információk összefoglalása

Ez az adatbázis egy iskolai büfék rendszer (MERN stack projekt) részét képezi, amely lehetővé teszi a felhasználók (diákok, szülők, tanárok) számára az étkezés megrendelését, kifizetését és értékelését. A rendszer támogatja a felhasználói autentikációt, a menükezelést, rendeléseket, kifizetéseket, hűségprogramokat és biztonsági naplózást. A fő cél az iskolai étkezés hatékony és biztonságos kezelése, beleértve a készletkezelést, értékeléseket és a pénzügyi tranzakciókat. Az adatbázis MongoDB-t használ Mongoose ODM-mel, amely egy NoSQL adatbázis, de sémákkal strukturált. A rendszer Redis-t használ gyorsítótárazáshoz a teljesítmény növelése érdekében.

Az adatbázis-modell típusa: NoSQL (MongoDB), lekérdezési nyelv: JavaScript (Mongoose queries). Kiegészítőként Redis in-memory adatbázis gyorsítótárazáshoz.

Adatbázis-terv és séma

Entitások és kapcsolatok (ER modell összefoglaló)

A rendszer fő entitásai és kapcsolataik:

- **User** (Felhasználó): Központi entitás, minden más entitáshoz kapcsolódik.
- **MenuItems** (Menüelemek): Étkezési tételek.
- **Order** (Rendelés): Felhasználók rendelései.
- **OrderItems** (Rendelés tételek): Egy rendeléshez tartozó Menüelemek.
- **Payment** (Kifizetés): Pénzügyi tranzakciók.
- **Review** (Értékelés): Menüelemek értékelése.
- **DailyMenu** (Napi menü): Iskolai periódusok szerinti menük.
- **ParentStudent** (Szülő-Diák kapcsolat): Szülők és diákok összekapcsolása.
- **SecurityLogs** (Biztonsági naplók): Események naplózása.
- **UserLoyalty** (Hűségprogram): Felhasználók pontjai, kedvezményei és hűség szintje.

Kapcsolatok:

- User 1:N Payment, Order, Review, SecurityLogs, UserLoyalty.
- User 1:N ParentStudent (szülőként vagy diákként).
- MenuItems 1:N OrderItems (beágyazott Order-ben), Review.
- Order 1:N OrderItems (beágyazott).
- DailyMenu 1:N MenuItems (referenciákon keresztül).

Nincs relációs adatbázis, így az ER diagram opcionális, de a kapcsolatok referenciákon alapulnak (ObjectId-k).

Relációs séma (táblázatok részletei)

Az alábbi táblázatokban minden entitás (kollekció) mezőit dokumentálom: név, típus, jelentés/szerep, megszorítások.

User (Felhasználók)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
username	String	Felhasználónév	Kötelező, egyedi
password	String	Jelszó (hash-elt)	Kötelező
email	String	E-mail cím	Kötelező, egyedi, e-mail formátum, trim
isVerified	Boolean	E-mail ellenőrzés státusza	Alapértelmezett: false
usertype	String	Felhasználó típusa (admin, student, parent, teacher, frozen)	Enum: ['admin', 'student', 'parent', 'teacher', 'frozen'], alapértelmezett: 'student'
createdAt	Date	Fiók létrehozási dátuma	Alapértelmezett: jelenlegi idő
balance	Number	Felhasználó egyenlege alkalmazáson belüli vásárlásokhoz	Alapértelmezett: 0

Üzleti szabályok: Minden felhasználónak egyedi felhasználóneve és e-mail címe van. A felhasználók típusa befolyásolja a hozzáférési jogokat (pl. admin mindenhez hozzáfér).

Payment (Kifizetések)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
userId	ObjectId (ref: User)	Fizető felhasználó	Opcionális
amount	Number	Fizetett összeg	Kötelező
currency	String	Pénznem (pl. USD, HUF)	Kötelező
paymentMethod	String	Fizetési mód	Kötelező
status	String	Státusz (Completed, Pending, Failed)	Kötelező, enum: ['Completed', 'Pending', 'Failed']
transactionId	String	Külső tranzakció referencia	Opcionális
createdAt	Date	Létrehozási idő	Alapértelmezett: jelenlegi idő

Üzleti szabályok: Minden kifizetés egy felhasználóhoz tartozik, de opcionális lehet (pl. vendég kifizetések).

Menulems (Menüelemek)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
name	String	Menüelem neve	Kötelező
description	String	Leírás	Kötelező
stock	Number	Készlet mennyisége	Kötelező, alapértelmezett: 0
price	Number	Ár	Kötelező
category	String	Kategória (Soup, Salad, stb.)	Kötelező, enum: ['Soup', 'Salad', 'MainDish', 'SideDish', 'Snack', 'Dessert', 'Drink', 'Healthy', 'SpecialDiet', 'DailySpecial', 'Other'], alapértelmezett: 'Other'
available	Boolean	Elérhetőség	Alapértelmezett: true
QRCode	String	QR kód a menüelemhez	Opcionális
allergens	[String]	Allergének listája	Alapértelmezett: []
nutritionallInfo.calories	Number	Kalóriák	Opcionális
nutritionallInfo.protein	Number	Fehérje	Opcionális
nutritionallInfo.carbs	Number	Szénhidrát	Opcionális
nutritionallInfo.fat	Number	Zsír	Opcionális

Üzleti szabályok: A készlet nem lehet negatív; kategóriák alapján szűrhető. Pre-save hook: Ha a készlet ≤ 0 , akkor available = false, különben true.

Order (Rendelések)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
userId	ObjectId (ref: User)	Rendelő felhasználó	Kötelező
items	[OrderItemsScheme]	Rendelés tételei	Kötelező
orderDate	Date	Rendelés dátuma	Alapértelmezett: jelenlegi idő
status	String	Státusz (Pending, InProgress, Completed, Cancelled)	Kötelező, enum: ['Pending', 'InProgress', 'Completed', 'Cancelled'], alapértelmezett: 'Pending'
totalAmount	Number	Teljes összeg	Kötelező
pickupTime	Date	Átvétel ideje	Opcionális
notes	String	Megjegyzések	Opcionális, alapértelmezett: "
paypalOrderId	String	PayPal rendelés azonosító	Opcionális
paymentMethod	String	Fizetési mód	Opcionális
transactionId	String	Tranzakció azonosító	Opcionális
publicID	String	Nyilvános azonosító	Kötelező, egyedi

Üzleti szabályok: Minden rendelés egy felhasználóhoz tartozik; státusz változások követik az üzleti folyamatot. Pre-save hook: Ha a rendelés 'Pending' státuszban van és több mint 15 perc telt el a létrehozás óta, automatikusan 'Cancelled'-re változik.

OrderItems (Rendelés tételek)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
menuItemId	ObjectId (ref: MenuItems)	Menüelem azonosító	Kötelező
orderId	ObjectId (ref: Order)	Rendelés azonosító	Opcionális
quantity	Number	Mennyiség	Kötelező, alapértelmezett: 1

Üzleti szabályok: Minden tétel egy menüelemhez tartozik; mennyiség pozitív egész szám. Ez a séma be van ágyazva az Order séma items mezőjébe.

Review (Értékelések)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
userId	ObjectId (ref: User)	Értékelő felhasználó	Kötelező
menuItemId	ObjectId (ref: MenuItems)	Értékelt menüelem	Kötelező
rating	Number	Értékelés (1-5)	Kötelező, min: 1, max: 5
comment	String	Megjegyzés	Opcionális
createdAt	Date	Létrehozási idő	Alapértelmezett: jelenlegi idő

Üzleti szabályok: Egy felhasználó többször is értékelhet különböző tételeket.

DailyMenu (Napi menü)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
date	Date	Dátum	Kötelező
schoolPeriod	String	Iskolai periódus (morning, afternoon)	Kötelező, enum: ['morning', 'afternoon']
menuItems	[ObjectId] (ref: MenuItem)	Menüelemek listája	Kötelező
createdAt	Date	Létrehozási idő	Alapértelmezett: jelenlegi idő

Üzleti szabályok: Napi menük periódusonként készülnek.

ParentStudent (Szülő-Diák kapcsolat)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
parentId	ObjectId (ref: User)	Szülő felhasználó	Kötelező
studentId	ObjectId (ref: User)	Diák felhasználó	Kötelező
createdAt	Date	Létrehozási idő	Alapértelmezett: jelenlegi idő

Üzleti szabályok: Szülők több diákhoz is kapcsolódhatnak.

SecurityLogs (Biztonsági naplók)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
userId	ObjectId (ref: User)	Felhasználó	Opcionális
action	String	Akció (pl. LOGIN_SUCCESS)	Kötelező
type	String	Típus (INFO, WARNING, ERROR)	Kötelező
ipAddress	String	IP cím (Hashelt)	Opcionális
Timestamp	Date	Időbélyeg	Alapértelmezett: jelenlegi idő
details	String	További információk	Opcionális
country	String	Ország	Opcionális
CountryCode	String	Országkód	Opcionális
currency	String	Pénznem	Opcionális
Continent	String	Kontinens	Opcionális
IsVPN	Boolean	VPN használat	Opcionális
isTor	Boolean	Tor használat	Opcionális
isProxy	Boolean	Proxy használat	Opcionális

Üzleti szabályok: Naplók minden fontos eseményt rögzítenek.

UserLoyalty (Hűségprogram)

Mező neve	Típus	Jelentés/Szerep	Megszorítások
userId	ObjectId (ref: User)	Felhasználó	Kötelező
totalPoints	Number	Összes pont	Alapértelmezett: 0
userTier	String	Felhasználó hűség szintje	Enum: ['none', 'Bronze', 'Silver', 'Gold', 'Platinum'], alapértelmezett: 'none'
discounts	String	Kedvezmények listája	[Lásd tábla alatt]:
lastUpdated	Date	Utolsó frissítés	Alapértelmezett: jelenlegi idő

Üzleti szabályok: Pontok vásárlások alapján gyűlnek. A hűség szint automatikusan frissül a pontok alapján (50 ponttól Bronze, 250-től Silver, 800-tól Gold, 2000-től Platinum). Pre-save hook: Tier frissítése a totalPoints alapján. Post-save hook: Ha a tier változott, új kedvezmények hozzáadása a tier alapján (Bronze: 5% healthy; Silver: 10% healthy, 5% drink 90 napig; Gold: 15% healthy, 10% full_meal; Platinum: 20% healthy, 15% general).

A discounts mező részletei (tömb elemei):

```
discounts: [{
  type: { type: String, enum: Object.values(DISCOUNT_TYPES), required: true }, // e.g., DISCOUNT_TYPES.HEALTHY
  rate: { type: Number, enum: Object.values(DISCOUNT_RATES), required: true }, // e.g., DISCOUNT_RATES.FIVE
  validUntil: { type: Date, required: false }
}],

const DISCOUNT_RATES = {
  FIVE: 0.05,
  TEN: 0.10,
  FIFTEEN: 0.15,
  TWENTY: 0.20,
  TWENTY_FIVE: 0.25,
};

const DISCOUNT_TYPES = {
  HEALTHY: 'healthy',
  VEGETARIAN: 'vegetarian',
  FULL_MEAL: 'full_meal',
  DRINK: 'drink',
  DESSERT: 'dessert',
  GENERAL: 'general',
};
```

Fizikai és logikai szerkezet

- **Táblák/Nézetek:** MongoDB kollekciók (collections) a fenti sémák alapján.
- **Indexek:** alapértelmezett indexek az `_id`-re és egyedi mezőkre (pl. username, email).
- **Tárolt eljárások/Függvények:** Nincs (JavaScript backend kezel mindent).
- **Gyorsítótárazás (Cache):** Redis in-memory adatbázis használata a teljesítmény növelésére, különösen a dashboard adatok gyors eléréséhez (pl. felhasználók listája, statisztikák), 5 perces lejárattal.

Használati esetek (Use Cases) és forgatókönyvek

- **Felhasználói regisztráció és autentikáció:** Diákok/szülők regisztrálnak, bejelentkeznek; adatok User kollekcióban.
- **Menü kezelése:** Admin hozzáadja/szerkeszti MenuItems-t; diákok böngézik DailyMenu alapján.
- **Rendelés leadása:** Diák kiválaszt tételeket OrderItems-ben, Order létrejön; Payment rögzíti kifizetést.
- **Értékelés:** Felhasználók Review-t adnak MenuItems-hez.
- **Hűségprogram:** Vásárlások után UserLoyalty frissül.
- **Biztonság:** Minden akció SecurityLogs-ban naplózódik.
- **Admin műveletek:** Felhasználók listázása, statisztikák (User, Order stb. alapján), Redis cache-ből gyorsítottn.

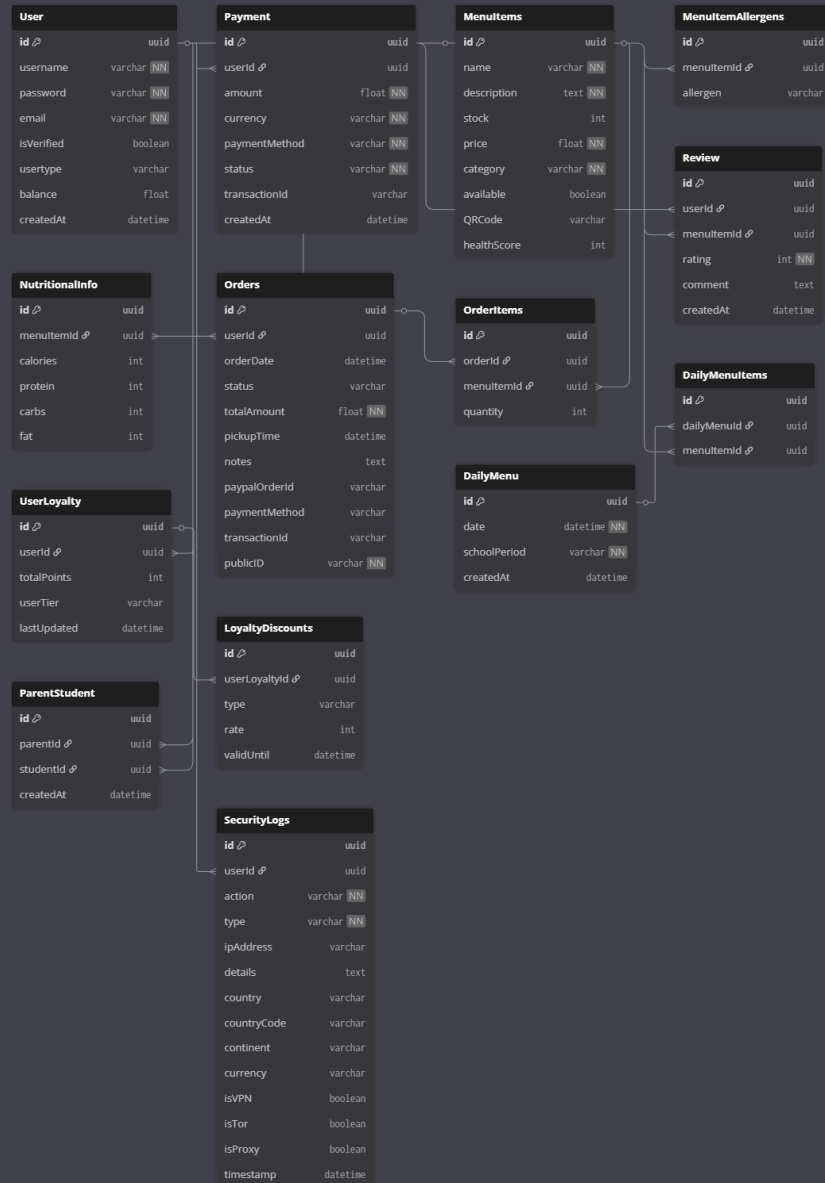
Biztonság és hozzáférés

- **Felhasználói szerepek:** Admin (teljes hozzáférés), Student/Parent/Teacher (korlátozott), Frozen (blokkolva).
- **Jogosultságok:** JWT tokenek, middleware-ek (pl. requireAdmin).
- **Adatvédelmi szabályok:** E-mail ellenőrzés, GDPR-kompatibilis (pl. személyes adatok védelme), IP cím GDPR kompatibilis tárolás.
- **Biztonság:** Jelszavak hash-elve (bcrypt), reCAPTCHA, IP naplózás, IP hashelés, VPN/Tor detektálás.

Karbantartás és üzemeltetés

- **Biztonsági mentési eljárások:** MongoDB dump/export rendszeres mentéshez; Redis esetében adatok ideiglenesek, így külön mentés nem szükséges.
- **Teljesítményfigyelés:** Lekérdezések optimalizálása, Redis cache használata dashboard-on a gyorsabb válaszidők érdekében.
- **Frissítési folyamatok:** Séma változásoknál migrációs szkriptek; verziókezelés Git-en keresztül. Redis konfiguráció környezeti változók alapján.

- **További:** Tesztelés (`database_testing.js`), kapcsolatkezelés környezeti változók alapján.



5.3 Algorithms and Data Structures

The SnapTray system employs various algorithms and data structures to ensure efficient data processing, security, and performance. This section documents the key algorithms and data structures used throughout the system.

5.3.1 Data Structures

5.3.1.1 MongoDB Collections (NoSQL Documents)

- **Document-based Storage:** MongoDB collections store data as BSON documents, allowing flexible schemas and nested structures.
- **Embedded Documents:** Order items are embedded within Order documents for atomic operations and better read performance.
- **References:** ObjectId references link related documents (e.g., User to Order, MenuItem to Review).
- **Arrays:** Used for storing multiple values like allergens, menu items in daily menus, and discount lists.

5.3.1.2 Redis Data Structures

- **Strings:** Session storage, cached user data, and simple key-value pairs.
- **Hashes:** Complex cached objects like user statistics and dashboard data.
- **Sorted Sets:** Rate limiting with score-based expiration.
- **Lists:** Queue structures for background processing (if implemented).

5.3.1.3 Redis Lua Scripting

Redis Lua scripts are used for atomic operations that require multiple Redis commands to execute as a single, uninterruptible transaction. This ensures data consistency in high-concurrency scenarios.

Key Lua Scripts in the System:

Order Processing Script (process_order.lua):

```
-- Atomic order processing with inventory management
local orderId = ARGV[1]
local userId = ARGV[2]
local items = cjson.decode(ARGV[3])

-- Check inventory availability
for i, item in ipairs(items) do
    local stock = redis.call('GET', 'item:' .. item.id .. ':stock')
    if not stock or tonumber(stock) < item.quantity then
        return redis.error_reply('INSUFFICIENT_STOCK')
    end
end

-- Deduct inventory atomically
for i, item in ipairs(items) do
    redis.call('DECRBY', 'item:' .. item.id .. ':stock', item.quantity)
end

-- Create order record
redis.call('HMSET', 'order:' .. orderId,
    'userId', userId,
    'status', 'CONFIRMED',
    'timestamp', redis.call('TIME')[1]
)

return redis.status_reply('ORDER_CONFIRMED')
```

- **Purpose:** Atomic inventory deduction and order creation
- **Benefits:** Prevents race conditions during high-traffic ordering
- **Atomicity:** All operations succeed or all fail together

Wallet Update Script (wallet_update.lua):

```
-- Atomic wallet balance updates with validation
local userId = ARGV[1]
local amount = tonumber(ARGV[2])
local operation = ARGV[3] -- 'add' or 'subtract'

local walletKey = 'wallet:' .. userId
local currentBalance = tonumber(redis.call('GET', walletKey) or '0')

if operation == 'subtract' and currentBalance < amount then
    return redis.error_reply('INSUFFICIENT_FUNDS')
end

local newBalance = operation == 'add' and (currentBalance + amount) or (currentBalance - amount)

redis.call('SET', walletKey, newBalance)
redis.call('PUBLISH', 'wallet_updates', userId .. ':' .. newBalance)

return redis.status_reply('BALANCE_UPDATED:' .. newBalance)
```

- **Purpose:** Thread-safe wallet balance modifications
- **Validation:** Prevents negative balances and insufficient funds
- **Notifications:** Publishes balance changes for real-time updates

Rate Limiting Script (rate_limit.lua):

```

-- Sliding window rate limiting
local key = KEYS[1]
local window = tonumber(ARGV[1]) -- window size in seconds
local limit = tonumber(ARGV[2]) -- max requests per window
local current = redis.call('TIME')[1]

-- Remove old entries outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, current - window)

-- Count current requests in window
local count = redis.call('ZCARD', key)

if count >= limit then
    return redis.error_reply('RATE_LIMIT_EXCEEDED')
end

-- Add current request
redis.call('ZADD', key, current, current)
redis.call('EXPIRE', key, window)

return redis.status_reply('ALLOWED:' .. (limit - count - 1) .. '_remaining')

```

- **Purpose:** Distributed rate limiting across multiple server instances
- **Algorithm:** Sliding window with sorted set for timestamp tracking
- **Accuracy:** More precise than fixed window but maintains performance

Lua Script Benefits:

- **Atomicity:** Multiple Redis operations execute as one atomic unit
- **Performance:** Reduces network round trips between application and Redis
- **Consistency:** Eliminates race conditions in concurrent operations

- **Reusability:** Scripts are cached on Redis server for repeated execution

5.3.1.4 JavaScript Objects and Arrays

- **Plain Objects:** Configuration objects, API responses, and in-memory data manipulation.
- **Arrays:** Cart management, menu filtering, and batch operations.
- **Maps:** Efficient key-value storage for caching and lookups.
- **Sets:** Unique value collections for filtering and deduplication.

5.3.2 Core Algorithms

5.3.2.1 Authentication and Security Algorithms

Password Hashing Algorithm:

```
// bcrypt with salt rounds for secure password storage
const hashedPassword = await bcrypt.hash(password, 12);
const isValid = await bcrypt.compare(password, hashedPassword);
```

- **Algorithm:** bcrypt with adaptive cost factor
- **Purpose:** Prevent rainbow table attacks and brute force attacks

JWT Token Generation and Verification:

```
// HS256 algorithm for token signing
const token = jwt.sign(payload, secretKey, { expiresIn: '24h' });
const decoded = jwt.verify(token, secretKey);
```

- **Algorithm:** HMAC-SHA256 (HS256)

- **Purpose:** Stateless authentication and secure data transmission
- **Security Features:** Expiration, issuer validation, audience restriction

IP Hashing for Privacy:

```
// SHA-256 hashing for GDPR compliance
const hashedIP = crypto.createHash('sha256').update(ipAddress).digest('hex');
```

- **Algorithm:** SHA-256 cryptographic hash
- **Purpose:** Anonymize IP addresses in logs while maintaining uniqueness
- **Compliance:** GDPR Article 32 data protection requirements

5.3.2.2 Payment Processing Algorithms

Currency Conversion Algorithm:

```
// Simple multiplication-based conversion
const convertCurrency = (amount, fromRate, toRate) => {
  return (amount / fromRate) * toRate;
};
```

- **Algorithm:** Linear conversion with exchange rates
- **Purpose:** Convert between different currencies for international payments
- **Precision:** Uses floating-point arithmetic with rounding to 2 decimal places

Payment Validation Algorithm:

```
// Multi-step validation with checksums
const validatePayment = (paymentData) => {
  return validateAmount(paymentData.amount) &&
    validateCurrency(paymentData.currency) &&
    validateChecksum(paymentData);
};
```

- **Algorithm:** Multi-condition validation with early termination
- **Purpose:** Prevent fraudulent transactions and data corruption
- **Error Handling:** Comprehensive validation with specific error messages

5.3.2.3 Loyalty Point Calculation Algorithm

Point Calculation Based on Purchase Amount:

```
const calculatePoints = (amount, tier, healthLevel, date) => {
  const basePoints = amount * 4; // 4 points per dollar
  const tierMultiplier = getTierMultiplier(tier);
  const healthBonus = healthLevel === 'HIGH' ? 1.5 : 1.0;
  const holidayBonus = isHoliday(date) ? 1.2 : 1.0;

  return Math.floor(basePoints * tierMultiplier * healthBonus * holidayBonus);
};
```

- **Algorithm:** Multiplicative point calculation with tier bonuses
- **Factors:** Purchase amount, user tier, health score, holiday periods
- **Rounding:** Floor function to ensure integer points

Tier Determination Algorithm:


```
const determineTier = (totalPoints) => {
  if (totalPoints >= 20000) return 'PLATINUM';
  if (totalPoints >= 8000) return 'GOLD';
  if (totalPoints >= 2500) return 'SILVER';
  if (totalPoints >= 1200) return 'BRONZE';
  return 'NONE';
};
```

- **Algorithm:** Threshold-based classification
- **Purpose:** Automatic tier promotion based on accumulated points
- **Triggers:** Real-time updates on point changes

5.3.2.4 Search and Filtering Algorithms

Menu Item Filtering Algorithm:

```
const filterMenuItems = (items, filters) => {
  return items.filter(item => {
    return (!filters.category || item.category === filters.category) &&
      (!filters.priceRange || isInRange(item.price, filters.priceRange)) &&
      (!filters.allergens || !hasAllergens(item, filters.allergens)) &&
      (!filters.searchTerm || matchesSearch(item, filters.searchTerm));
  });
};
```

- **Algorithm:** Multi-criteria filtering with short-circuit evaluation
- **Purpose:** Efficient menu browsing with multiple filter options

Text Search Algorithm:

```
const matchesSearch = (item, term) => {  
  const searchTerm = term.toLowerCase();  
  return item.name.toLowerCase().includes(searchTerm) ||  
    item.description.toLowerCase().includes(searchTerm);  
};
```

- **Algorithm:** Case-insensitive substring matching
- **Purpose:** Basic text search across menu items
- **Limitations:** Simple implementation, could be enhanced with fuzzy matching

5.3.2.5 Caching Algorithms

Cache Key Generation:

```
const generateCacheKey = (userId, resource, params) => {  
  return `${userId}:${resource}:${JSON.stringify(params)}`;  
};
```

- **Algorithm:** Structured key generation with JSON serialization
- **Purpose:** Unique cache keys for different user-resource combinations
- **Collision Prevention:** Includes all relevant parameters

Cache Invalidation Strategy:

```
const invalidateUserCache = async (userId) => {
  const pattern = `${userId}:*`;
  const keys = await redisClient.keys(pattern);
  if (keys.length > 0) {
    await redisClient.del(keys);
  }
};
```

- **Algorithm:** Pattern-based bulk invalidation
- **Purpose:** Clear all user-related cache entries on data changes
- **Performance:** Uses Redis pattern matching for efficiency

5.3.2.6 Rate Limiting Algorithm

Token Bucket Algorithm (via express-rate-limit):

```
// Implemented through express-rate-limit middleware
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  store: new RedisStore({ /* redis config */ })
});
```

- **Algorithm:** Token bucket with fixed window
- **Purpose:** Prevent abuse and ensure fair resource usage
- **Storage:** Redis-backed for distributed rate limiting

5.3.2.7 Data Validation Algorithms

Input Sanitization Algorithm:

```
const sanitizeInput = (input) => {  
  return input  
    .trim()  
    .replace(/[<>]/g, '') // Basic XSS prevention  
    .substring(0, 1000); // Length limiting  
};
```

- **Algorithm:** Multi-step sanitization with regex replacement
- **Purpose:** Prevent injection attacks and normalize input data
- **Layers:** Client-side, server-side, and database-level validation

Email Validation Algorithm:

```
const validateEmail = (email) => {  
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  return emailRegex.test(email) && email.length <= 254;  
};
```

- **Algorithm:** Regular expression matching with length constraints
- **Purpose:** Ensure valid email format and prevent buffer overflow

5.3.3 Performance Optimization Algorithms

5.3.3.1 Database Query Optimization

Index Utilization:

```
// Compound indexes for common query patterns
User.collection.createIndex({ email: 1, isVerified: 1 });
Order.collection.createIndex({ userId: 1, orderDate: -1 });
```

- **Algorithm:** Strategic index creation based on query patterns
- **Purpose:** Reduce query execution time
- **Maintenance:** Automatic index updates on document changes

5.3.3.2 Pagination Algorithm

Cursor-based Pagination:

```
const getPaginatedResults = async (model, filter, page, limit) => {
  const skip = (page - 1) * limit;
  const results = await model.find(filter)
    .sort({ createdAt: -1 })
    .skip(skip)
    .limit(limit);
  const total = await model.countDocuments(filter);
  return { results, total, page, pages: Math.ceil(total / limit) };
};
```

- **Algorithm:** Skip-limit pagination with total count
- **Purpose:** Efficient handling of large datasets

5.3.3.3 Batch Processing Algorithm

Order Fulfillment Batch Processing:

```
const processOrderBatch = async (orders) => {
  const bulkOps = orders.map(order => ({
    updateOne: {
      filter: { _id: order._id },
      update: { status: 'Completed' }
    }
  }));
  return await Order.bulkWrite(bulkOps);
};
```

- **Algorithm:** Bulk database operations for multiple records
- **Purpose:** Reduce database round trips for batch updates

5.3.4 Security Algorithms

5.3.4.1 reCAPTCHA Verification

Score-based Assessment:

```
const verifyRecaptcha = async (token) => {
  const response = await fetch('https://www.google.com/recaptcha/api/siteverify', {
    method: 'POST',
    body: new URLSearchParams({
      secret: process.env.RECAPTCHA_SECRET,
      response: token
    })
  });
  const result = await response.json();
  return result.score >= 0.5; // Threshold for human/bot determination
};
```

- **Algorithm:** Machine learning-based risk assessment

- **Purpose:** Distinguish between human users and automated bots
- **Accuracy:** Google's proprietary algorithm with score 0.0-1.0 (1 being the highest chance the user is a human and 0 being the lowest chance, 0.5 the default threshold for human detection)

5.3.4 Algorithm Selection Rationale

- **Security Algorithms:** Chosen for industry-standard security (bcrypt, JWT, AES)
- **Performance Algorithms:** Selected for scalability (Redis caching, pagination, indexing)
- **Business Logic Algorithms:** Designed for fairness and transparency (loyalty calculations, tier systems)
- **Data Processing Algorithms:** Optimized for common use cases (filtering, searching, validation)

The algorithms are chosen to balance security, performance, maintainability, and user experience while adhering to industry best practices and compliance requirements.

5.4 Security Design

5.4.1 Security Features

- **Authentication:** JWT-based authentication with role-based access control (RBAC).
- **Data Encryption:** Sensitive data encrypted at rest and in transit (TLS/SSL).
- **Input Validation:** Comprehensive validation and sanitization of all user inputs.
- **Rate Limiting:** Prevent brute-force attacks using express-rate-limit with Redis store and Redis rate limiting via Lua integration.
- **Logging and Monitoring:** SecurityLogs collection for tracking user actions and potential security incidents.
- **Regular Audits:** Periodic security audits and vulnerability assessments.
- **Backup and Recovery:** Regular backups of the database and secure storage of backup files. NOT YET IMPLEMENTED

- **Compliance:** Adherence to GDPR and other relevant data protection regulations.
- **XSS Protection:** Use of libraries like *xss-clean* and *helmet* to mitigate XSS attacks.
- **HTTP Pollution Protection:** Use of *helmet* to set secure HTTP headers.
- **CSRF Protection:** Implementation of CSRF tokens for state-changing operations to prevent cross-site request forgery attacks. NOT YET IMPLEMENTED
- **Password Policies:** Enforcing strong password requirements..
- **Two-Factor Authentication (2FA):** Optional 2FA for enhanced security. NOT YET IMPLEMENTED
- **Session Management:** Secure session handling with appropriate expiration and invalidation.
- **Proxy/VPN/Tor Detection:** Logging and potential blocking of suspicious IPs using third-party services. NOT YET IMPLEMENTED
- **reCAPTCHA Integration:** To prevent automated bot interactions during registration and login.
- **IP Hashing:** Hashing IP addresses before storage to enhance user privacy.
- **NoSQL Injection Prevention:** Use of parameterized queries and ODM features and libraries to prevent injection attacks.
- **Security Headers:** Implementation of security headers using Helmet.js to protect against common vulnerabilities.
- **Content Security Policy (CSP):** Define and enforce a strict CSP to mitigate XSS and data injection attacks.
- **Detailed CSP Configuration:** The application implements a comprehensive CSP that restricts resource loading to approved domains only. This includes restrictions on script sources, style sources, image sources, and form actions. Inline scripts are prohibited except for specific nonces, and `eval()` is completely disabled.

5.4.2 Security Policies

- **Access Control:** Strict role-based access control (RBAC) to limit user permissions.
- **Data Retention:** Policies for data retention and deletion in compliance with regulations.

- **Incident Response:** Procedures for responding to security incidents and breaches.
- **User Education:** Informing users about security best practices.
- **Regular Updates:** Keeping software and dependencies up to date with security patches.

5.4.3 In depth Security Measures

- **Password Hashing:** All passwords are hashed using bcrypt with a salt of 12 rounds when registering and 10 rounds when resetting password. VERIFY
- JWT Authentication:** JSON Web Tokens (JWT) are used for stateless authentication between the client and server. JWT signing secrets are generated using cryptographically secure random values stored in environment variables. This approach ensures that token secrets are unpredictable and resistant to attacks. Standard JavaScript functions like Math.random(), which rely on algorithms such as xorshift128+, are not suitable for cryptographic purposes due to their predictability, highlighting the importance of using a secure random number generator for authentication secrets.
- **Payment Security:** Integration with PayPal's and Google Pay's secure payment gateways, ensuring PCI compliance they are encrypting payment data via AES-256 encryption.
- **XSS Protection:** Use of libraries like *xss-clean* and *helmet* to mitigate XSS attacks by sanitizing user inputs and setting secure HTTP headers.
- **CSRF Protection:** Implementation of CSRF tokens for state-changing operations to prevent cross-site request forgery attacks.
- **Rate Limiting:** The site has 2 different rate limiting methods, one via *express-rate-limit* with *Redis* store to limit requests per IP per time window, and another via Redis Lua scripting for the site's dashboard routes to prevent abuse and ensure fair resource usage.
- **IP Hashing:** IP addresses are hashed using SHA-256 before storage in SecurityLogs to enhance user privacy while maintaining the ability to track unique IPs.
- **reCAPTCHA Integration:** Google reCAPTCHA v3 is integrated into the registration and login processes to prevent automated bot interactions, using a score-based assessment to differentiate between human users and bots. The reCAPTCHA secret key is securely

stored in environment variables. It uses Google's proprietary machine learning algorithm to assign a score between 0.1 and 1.0, with a threshold of 0.5 for human detection this helps to reduce spam and abuse on the platform.

- **NoSQL Injection Prevention:** The application uses parameterized queries and ODM features to prevent NoSQL injection attacks, ensuring that user inputs are properly sanitized and validated before being used in database operations. It also uses libraries like *express-mongo-sanitize* to further protect against injection attacks.
- **IP Detection** The site uses a third-party service (iplocate.io) to detect if an IP address is using a VPN, Proxy, or Tor network. This information is logged in the SecurityLogs collection for monitoring purposes and potential blocking of suspicious IPs. The service provides details such as country, country code, continent, and whether the IP is associated with VPN, Proxy, or Tor usage. The free tier of the service currently let's us have 1k requestes/day NOT YET IMPLEMENTED FAR
- **HTTP Parameter Pollution (HPP)** The application uses the *hpp* library middleware to protect against HTTP Parameter Pollution attacks by sanitizing query parameters and ensuring that only the first occurrence of a parameter is considered.
- **CORS Policy:** The application implements a strict CORS policy using the *cors* middleware to control which domains can access the API, preventing unauthorized cross-origin requests, this is configured to only allow requests from the official frontend domain
- **Security Headers:** The application uses Helmet.js to set various HTTP headers that enhance security, such as Content Security Policy (CSP), X-Content-Type-Options, X-Frame-Options, and others to protect against common web vulnerabilities.
- **Database Security:** MongoDB connections use authentication with credentials stored in environment variables. The application implements connection pooling and timeout configurations to prevent resource exhaustion attacks. Database queries use Mongoose ODM which provides built-in protection against NoSQL injection through schema validation and type casting.
- **Error Handling Security:** Sensitive information is never exposed in error messages sent to clients. Error responses are sanitized and provide minimal information to prevent information leakage that could aid attackers.
- **API Security:** All API endpoints require authentication by default. Public endpoints (like registration) are explicitly marked and protected with additional rate limiting and bot detection.

- **Environment Security:** All sensitive configuration (API keys, database credentials, JWT secrets) are stored in environment variables and never committed to version control. The application validates required environment variables on startup.

5.4.4 Security Testing and Validation

- **Automated Security Testing:** The project includes security-focused test suites that validate authentication flows, input sanitization, and rate limiting effectiveness.
- **Penetration Testing:** Periodic manual security assessments to identify vulnerabilities not caught by automated tools.

6. Implementation

6.1 Directory Structure

```
├─ .env
├─ .git/
├─ .gitattributes
├─ .github/
├─ .gitignore
├─ .idea/
├─ .vscode/
├─ code_analytics.json      # Code analytics data
├─ config/                  # Configuration files
│   ├── DATABASE_CONSTANTS.JS
│   ├── database_queries.js
│   └─ hu.json
├─ data/                    # Data files
│   ├── disposable_email_list.json
│   ├── Most_used_passwords.json
│   ├── password_characters.json
│   └─ database_test/      # Test database files
│       ├── Food_Items.json
│       └─ menu_items.json
├─ docs/                    # Documentation
│   ├── database.png
│   ├── DatabaseDoc.md
│   ├── DatabaseDoc.pdf
│   ├── Documentation.html
│   ├── Documentation.md
│   ├── Documentation.pdf
│   ├── Paypal_TestDetails.txt
│   ├── RedisLua_README.md
│   └─ snaptraySTACK.png
```

```
|   └─ sourcefor_security_checks.txt
|─ node_modules/
|─ package-lock.json
|─ package.json          # Node.js dependencies and scripts
|─ postcss.config.js     # PostCSS configuration
|─ public/              # Static files served to client (Frontend)
|   └─ favicon.ico
|   └─ googlepay.js
|   └─ index.html
|   └─ password_reset.html
|   └─ pay.html
|   └─ paypal.js
|   └─ register.html
|   └─ verify.html
|   └─ 404/              # 404 error pages
|       └─ 404.html
|       └─ 404.jsx
|   └─ 429/              # 429 error pages
|       └─ 429.html
|       └─ 429.jsx
|   └─ dashboard/        # Dashboard pages
|       └─ admin/
|           └─ admin.html
|           └─ admin.jsx
|           └─ AdminHeader.jsx
|           └─ AdminSidebar.jsx
|           └─ MenuItemsSection.jsx
|           └─ SettingsSection.jsx
|           └─ StatsSection.jsx
|           └─ useAdminData.js
|           └─ UsersSection.jsx
|       └─ parent/
|           └─ parent.html
|           └─ parent.jsx
|           └─ ParentHeader.jsx
```

- | | | |─ ParentOrdersSection.jsx
- | | | |─ ParentSettingsSection.jsx
- | | | |─ ParentSidebar.jsx
- | | | |─ ParentStatsSection.jsx
- | | | |─ ParentStudentsSection.jsx
- | | | |─ useParentData.js
- | | |─ student/
 - | | | |─ components/
 - | | | |─ hooks/
 - | | | |─ LoyaltySection.jsx
 - | | | |─ OrdersSection.jsx
 - | | | |─ services/
 - | | | |─ SettingsSection.jsx
 - | | | |─ StatsSection.jsx
 - | | | |─ student.html
 - | | | |─ student.jsx
 - | | | |─ StudentHeader.jsx
 - | | | |─ StudentSidebar.jsx
 - | | | |─ TransactionsSection.jsx
 - | | | |─ useStudentData.js
 - | | | |─ utils/
 - | | | |─ WalletSection.jsx
- | |─ home_page/ # Home page files
 - | | |─ home_page.html
 - | | |─ home_page.jsx
- | |─ information/ # Information pages
 - | | |─ index.html
 - | | |─ information.jsx
- | |─ no_perm/ # No permission pages
 - | | |─ index.html
 - | | |─ no_perm.jsx
- | |─ Order/ # Order pages
 - | | |─ Cart.jsx
 - | | |─ Header.jsx
 - | | |─ index.html

```
|   |   |─ LoyaltyStatus.jsx
|   |   |─ MenuItem.jsx
|   |   |─ notifications.js
|   |   |─ order.jsx
|   |   |─ paymentHandlers.js
|   |   |─ useCart.js
|   |─ readme.md                # Project documentation
|   |─ src/                     # Server-side source code (backend)
|   |   |─ admin/
|   |   |   |─ admin.js
|   |   |─ api.js
|   |   |─ auth/                # Authentication modules
|   |   |   |─ 2fa.js
|   |   |   |─ email_verification.js
|   |   |   |─ index.js
|   |   |   |─ login.js
|   |   |   |─ middleware.js
|   |   |   |─ password_reset.js
|   |   |   |─ passwordhash.js
|   |   |   |─ register.js
|   |   |   |─ security.js
|   |   |   |─ validation.js
|   |   |─ chapta.js
|   |   |─ dashboard/
|   |   |   |─ admin/
|   |   |   |   |─ admin.js
|   |   |   |─ dashboard.js
|   |   |   |─ middleware/
|   |   |   |   |─ auth-middleware.js
|   |   |   |─ parent/
|   |   |   |   |─ parent.js
|   |   |   |─ services/
|   |   |   |   |─ cache-service.js
|   |   |   |─ statistics/
|   |   |   |   |─ statistics.js
```

```
| | └─ student/
| |   └─ student.js
| └─ database.js
| └─ examples/
| |   └─ lua-demo.js
| └─ logout.js
| └─ LoyaltySystem/
| |   └─ loyalty-service.js
| └─ main.js
| └─ middleware/
| |   └─ security.js
| └─ models/
| |   └─ User.js
| └─ Orders/
| |   └─ Order.js
| └─ payments/
| |   └─ googlepay.js
| |   └─ paypal.js
| └─ redis-lua.js
| └─ redis.js
| └─ Register.jsx
| └─ script-loader.js
| └─ scripts/
| |   └─ process_order.lua
| |   └─ rate_limit.lua
| |   └─ TODO.md
| |   └─ wallet_update.lua
| └─ services/          # Service modules
| |   └─ googlepay-service.js
| |   └─ order-service.js
| |   └─ paypal-service.js
| |   └─ redis-lua-service.js
| └─ verificationStore.js
└─ tailwind.config.js    # Tailwind CSS configuration
└─ tests/                # Test files
```



```
├─ code_analytic.py
├─ code_analytics.json
├─ creating_test_users.js
├─ database_testing.js
├─ fake_data.py
├─ menu_items.json
├─ Paypal_TestConfig.txt
├─ query_security_logs.js
├─ register_testing.py
├─ Jest/                # Jest test directory
└─ performance_tests/   # Performance test files
    ├─ artillery.yml
    └─ reports/
        ├─ 20260121_916.txt
        └─ 20260121_936.txt
```

6.2 Backend Implementation

6.2.1 Technology Stack

The backend of the SnapTray system is built using a robust and scalable technology stack designed to handle high traffic, ensure data integrity, and provide a secure environment for users. The project leverages modern technologies and best practices to deliver a reliable service. The stack is used as follows:

- **Runtime Environment:** Node.js
- **Web Framework:** Express.js
- **Database:** MongoDB (NoSQL)
- **In-Memory Data Store:** Redis
- **Authentication:** JSON Web Tokens (JWT)

- **Payment Gateways:** PayPal, Google Pay
- **Caching:** Redis with Lua scripting for atomic operations

Express.js was chosen for its minimalistic and flexible nature, allowing for rapid development and easy integration with various middleware. MongoDB provides a schema-less design that accommodates the dynamic nature of the application's data, while Redis enhances performance through caching and supports complex operations via Lua scripting.

7. Testing and Validation

8. User Manual

9. Deployment and Maintenance

10. Conclusion and Future Work

11. References

12. Appendices
